

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL VII
POLYMORPHISM



Disusun Oleh:
Arif Mufti Tharsa 105224041

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Di THT Modul 5 ini, saya membuat program Java bernama Main.java yang mensimulasikan sistem pengelolaan layanan pengiriman paket pada sebuah perusahaan ekspedisi. Program ini terdiri dari satu parent class yaitu LayananEkspedisi dan tiga child class yaitu LayananReguler, LayananExpress, dan LayananInternasional. Materi utama yang diterapkan adalah Polymorphism, yaitu kemampuan satu method untuk berperilaku berbeda tergantung pada tipe objek yang memanggilnya, yang diwujudkan melalui method overriding dan instanceof casting.

Secara umum, program ini dirancang untuk menghitung ongkos kirim sesuai jenis layanan masing-masing, di mana setiap child class mengimplementasikan hitungOngkir() dengan logika yang berbeda-beda. Semua objek layanan disimpan dalam satu ArrayList bertipe parent class dan diproses dalam satu loop yang sama, namun Java secara otomatis memanggil versi method yang tepat berdasarkan tipe objek aslinya. Selain itu, program juga menerapkan method overloading pada LayananReguler dan fitur khusus seperti klaim asuransi pada LayananExpress serta cetak manifest pada LayananInternasional.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nomorResi	String	Menyimpan nomor resi unik setiap paket pengiriman (atribut LayananEkspedisi)
2	beratAktualKg	double	Menyimpan berat aktual paket dalam satuan kilogram (atribut LayananEkspedisi)
3	panjang	double	Menyimpan dimensi panjang paket dalam satuan cm (atribut LayananEkspedisi)
4	lebar	double	Menyimpan dimensi lebar paket dalam satuan cm (atribut LayananEkspedisi)
5	tinggi	double	Menyimpan dimensi tinggi paket dalam satuan cm (atribut LayananEkspedisi)
6	negaraTujuan	String	Menyimpan nama negara tujuan pengiriman (atribut LayananInternasional)
7	nilaiBarangUSD	double	Menyimpan nilai deklarasi barang dalam dolar AS untuk perhitungan pajak (atribut LayananInternasional)
8	daftar	ArrayList <LayananEkspedisi>	Menyimpan semua objek layanan pengiriman yang akan diproses di Main
9	totalPendapatanPerusahaan	double	Mengakumulasikan total ongkir dari semua layanan sebagai pendapatan perusahaan di Main

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	LayananEkspedisi(String, double, double, double, double)	Constructor	Menginisialisasi semua atribut dasar paket: nomorResi, beratAktualKg, panjang, lebar, dan tinggi
2	hitungBeratEfektif()	Functional	Menghitung berat volumetrik ($p \times l \times t / 6000$) lalu mengembalikan nilai terbesar antara berat aktual dan volumetrik
3	cetakResi()	Procedural	Mencetak nomor resi dan berat efektif paket ke konsol
4	hitungOngkir()	Functional	Method dasar yang mengembalikan 0.0; dirancang untuk di-override oleh setiap child class
5	LayananReguler(String, double, double, double, double)	Constructor	Membuat objek layanan reguler dengan memanggil constructor parent lewat super()
6	hitungOngkir() [Override]	Functional	Menghitung ongkir reguler: berat efektif dikali Rp15.000
7	hitungOngkir(boolean, int)	Functional (overload)	Menghitung ongkir reguler dengan diskon 10% jika member, ditambah biaya jarak Rp500 per km
8	LayananExpress(String, double, double, double, double)	Constructor	Membuat objek layanan express dengan memanggil constructor parent lewat super()
9	hitungOngkir() [Override]	Functional	Menghitung ongkir express: berat efektif dikali Rp30.000
10	klaimAsuransi(double)	Procedural	Memproses klaim asuransi; jika nilai barang di atas Rp1.000.000 maka diproses prioritas VIP
11	LayananInternasional(String, double, double, double, double, String, double)	Constructor	Membuat objek layanan internasional; memanggil super() lalu mengatur negaraTujuan dan nilaiBarangUSD
12	hitungOngkir() [Override]	Functional	Menghitung ongkir internasional: berat efektif dikali Rp200.000, ditambah pajak 20% jika nilai barang di atas \$50
13	cetakManifest()	Procedural	Mencetak manifest pengiriman internasional berisi negara tujuan dan nilai deklarasi barang

IV. Dokumentasi dan Pembahasan Code

Berikut adalah kode lengkap program beserta penjelasan tiap bagiannya:

LayananEkspedisi.java

```
public class LayananEkspedisi {
    protected String nomorResi;
    protected double beratAktualKg, panjang, lebar, tinggi;

    public LayananEkspedisi(String nomorResi, double beratAktualKg, double panjang,
double lebar, double tinggi) {
        this.nomorResi = nomorResi;
        this.beratAktualKg = beratAktualKg;
        this.panjang = panjang;
        this.lebar = lebar;
        this.tinggi = tinggi;
    }

    public double hitungBeratEfektif() {
        double volumetrik = (panjang * lebar * tinggi) / 6000;
        return Math.max(beratAktualKg, volumetrik);
    }

    public void cetakResi() {
        System.out.println("Nomor Resi: " + nomorResi);
        System.out.println("Berat Efektif: " + hitungBeratEfektif() + " Kg");
    }

    public double hitungOngkir() {
        return 0.0;
    }
}
```

LayananReguler.java

```
public class LayananReguler extends LayananEkspedisi {

    public LayananReguler(String nomorResi, double beratAktualKg, double panjang,
double lebar, double tinggi) {

        super(nomorResi, beratAktualKg, panjang, lebar, tinggi);
    }

    @Override
    public double hitungOngkir() {

        return hitungBeratEfektif() * 15000;
    }

    public double hitungOngkir(boolean isMember, int jarakKm) {

        double total = hitungOngkir();
    }
}
```

```

        if(isMember) {

            total = total - (total * 0.1);

        }

        total = total + (500 * jarakKm);

        return total;

    }

}

```

LayananExpress.java

```

public class LayananExpress extends LayananEkspedisi {

    public LayananExpress(String nomorResi, double beratAktualKg, double panjang,
double lebar, double tinggi) {

        super(nomorResi, beratAktualKg, panjang, lebar, tinggi);

    }

    @Override

    public double hitungOngkir() {

        return hitungBeratEfektif() * 30000;

    }

    public void klaimAsuransi(double nilaiBarang) {

        if(nilaiBarang > 1000000) {

            System.out.println("Klaim Asuransi VIP Rp" + nilaiBarang + " untuk resi " +
nomorResi + " sedang diproses prioritas.");

        } else {

            System.out.println("Klaim Asuransi Standar diproses dalam 7 hari kerja.");

        }

    }

}

```

LayananInternasional.java

```

public class LayananInternasional extends LayananEkspedisi {

    private String negaraTujuan;

```

```

private double nilaiBarangUSD;

    public LayananInternasional(String nomorResi, double beratAktualKg, double
panjang, double lebar, double tinggi, String negaraTujuan, double nilaiBarangUSD) {

        super(nomorResi, beratAktualKg, panjang, lebar, tinggi);

        this.negaraTujuan = negaraTujuan;

        this.nilaiBarangUSD = nilaiBarangUSD;

    }

    @Override

    public double hitungOngkir() {

        double ongkirDasar = hitungBeratEfektif() * 200000;

        double pajak = 0;

        if(nilaiBarangUSD > 50) {

            pajak = ongkirDasar * 0.2;

        }

        return ongkirDasar + pajak;

    }

    public void cetakManifest() {

        System.out.println("Manifest Internasional ke " + negaraTujuan + " - Deklarasi
Nilai: $" + nilaiBarangUSD);

    }

}

```

Main.java

```

import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {

        ArrayList<LayananEkspedisi> daftar = new ArrayList<>();

        daftar.add(new LayananReguler("REG-11", 2, 50, 50, 50));

        daftar.add(new LayananExpress("EXP-22", 5, 10, 10, 10));

    }

}

```

```
daftar.add(new LayananInternasional("INT-33", 3, 20, 20, 20, "Korea", 100));

double totalPendapatanPerusahaan = 0.0;
for(LayananEkspedisi layanan : daftar) {

    layanan.cetakResi();

    totalPendapatanPerusahaan += layanan.hitungOngkir();

    if(layanan instanceof LayananReguler) {

        LayananReguler reguler = (LayananReguler) layanan;

        System.out.println(

            "Ongkir Member + Jarak: Rp" +

            reguler.hitungOngkir(true, 25)

        );

    }

    if(layanan instanceof LayananExpress) {

        LayananExpress express = (LayananExpress) layanan;

        express.klaimAsuransi(1500000);

    }

    if(layanan instanceof LayananInternasional) {

        LayananInternasional internasional = (LayananInternasional) layanan;

        internasional.cetakManifest();

    }

    System.out.println();

}

System.out.println(

    "Total Pendapatan Perusahaan: Rp" +

    totalPendapatanPerusahaan

);

}
```


1. Class LayananEkspedisi (Parent Class)

LayananEkspedisi adalah parent class yang menjadi fondasi dari seluruh program. Atributnya dideklarasikan sebagai protected agar bisa diakses langsung oleh child class tanpa perlu getter. Method hitungBeratEfektif() menghitung berat volumetrik dari dimensi paket menggunakan rumus standar ekspedisi ($p \times l \times t$ dibagi 6000), lalu membandingkannya dengan berat aktual menggunakan Math.max() sehingga yang dipakai selalu yang lebih besar. Method hitungOngkir() di parent class sengaja dikosongkan dan mengembalikan 0.0, karena setiap child class wajib mengimplementasikan logikanya sendiri.

2. Class LayananReguler (Child Class)

LayananReguler meng-override hitungOngkir() dengan tarif Rp15.000 per kg berat efektif. Selain itu, class ini juga menerapkan method overloading dengan versi hitungOngkir(boolean, int) yang menerima status member dan jarak tempuh sebagai parameter. Jika pelanggan adalah member, ongkir dasar dipotong 10%, kemudian ditambahkan biaya jarak sebesar Rp500 per km. Overloading di sini memungkinkan satu nama method menangani dua skenario yang berbeda tanpa harus membuat nama baru.

3. Class LayananExpress (Child Class)

LayananExpress meng-override hitungOngkir() dengan tarif dua kali lipat dari reguler, yaitu Rp30.000 per kg berat efektif, mencerminkan prioritas pengiriman yang lebih cepat. Class ini juga punya method eksklusif klaimAsuransi(double) yang hanya dimiliki LayananExpress. Method ini mengecek nilai barang: jika di atas Rp1.000.000 maka klaim diproses sebagai VIP dengan prioritas, sedangkan di bawahnya diproses standar dalam 7 hari kerja.

4. Class LayananInternasional (Child Class)

LayananInternasional punya dua atribut tambahan yaitu negaraTujuan dan nilaiBarangUSD yang tidak ada di parent class. Logika hitungOngkir()-nya paling kompleks: ongkir dasar dihitung dari berat efektif dikali Rp200.000, lalu jika nilai barang melebihi \$50 akan dikenakan pajak tambahan sebesar 20% dari ongkir dasar. Method cetakManifest() adalah fitur eksklusif class ini untuk mencetak dokumen manifest yang diperlukan dalam proses pengiriman ke luar negeri.

5. Polymorphism dan instanceof di Main

Di class Main, semua objek layanan disimpan dalam ArrayList<LayananEkspedisi> dan diproses dalam satu enhanced for loop. Saat layanan.hitungOngkir() dipanggil, Java secara otomatis menjalankan versi yang sesuai dengan tipe objek aslinya, inilah inti dari polymorphism runtime. Untuk mengakses method eksklusif masing-masing child class seperti hitungOngkir(boolean, int), klaimAsuransi(), dan cetakManifest(), program menggunakan instanceof untuk mengecek tipe objek, lalu melakukan downcasting ke child class yang tepat sebelum method tersebut bisa dipanggil.

V. Kesimpulan

Polymorphism runtime membuat program lebih efisien karena semua objek dapat disimpan dalam ArrayList bertipe parent class dan method hitungOngkir() dapat dipanggil dalam satu loop meskipun setiap layanan memiliki logika perhitungan yang berbeda. Hal ini didukung oleh method overriding, di mana setiap child class mengimplementasikan hitungOngkir() sesuai aturan dan tarif masing-masing, sementara parent class hanya menyediakan kerangka dasarnya. Dengan konsep ini, program menjadi lebih fleksibel dan mudah dikembangkan karena penambahan layanan baru cukup dilakukan dengan membuat child class baru tanpa mengubah kode yang sudah ada.

Selain itu, penggunaan instanceof dan downcasting diperlukan untuk mengakses method khusus milik child class yang tidak tersedia di parent class, sehingga pengecekan tipe objek perlu dilakukan agar terhindar dari ClassCastException. Method overloading juga menunjukkan penerapan polymorphism dalam satu class, yaitu dengan menyediakan beberapa method bernama sama tetapi memiliki parameter berbeda sehingga kode menjadi lebih rapi, fleksibel, dan mudah digunakan.